

Negociacion TLS y flujo MQTT: conexion, publicacion, retencion y recepcion

Enfoque exclusivo en transporte TCP/TLS y capa MQTT — Maestro_TLS_Principal.py <-> Esclavo_TLS_KEYSTREAM.py (sin difusion/confusion/descifrado)

CLIENTE MQTT — MAESTRO (emisor)

Script: Maestro_TLS_Principal.py
CA local:
/etc/mosquitto/ca_certificates/ca.crt
tls_set(ca_certs=...,
tls_version=PROTOCOL_TLS_CLIENT)

BROKER MQTT — Mosquitto (TLS)

Host: raspberrypiJED.local Puerto: 8883
Posee: server.crt + server.key (propios)
Autenticacion MQTT: password_file (usuario1)

CLIENTE MQTT — ESCLAVO (receptor)

Script: Esclavo_TLS_KEYSTREAM.py
CA local:
/home/tunchi/CifradoSlave/certs/ca.crt
tls_set(ca_certs=...,
tls_version=PROTOCOL_TLS_CLIENT)

Ficha de archivos criptograficos involucrados

Archivo	Secreto	Ubicacion	Funcion
ca.crt	No	Maestro: /etc/mosquitto/ca_certificates/ca.crt - Esclavo: /home/tunchi/CifradoSlave/certs/ca.crt	Certificado publico y autofirmado de una CA local (subject=issuer, CN=MQTT-Local-CA; vigente 2025-2035 segun el ca.crt incluido en el repositorio). Cada cliente lo usa para verificar la firma del certificado que presenta el broker.
server.crt	No	Solo en el broker (Raspberry Pi) — ruta habitual /etc/mosquitto/certs/server.crt	Certificado publico del broker: contiene su llave publica y el SAN=raspberrypiJED.local; firmado por la CA anterior. Se envia al cliente durante el handshake (mensaje Certificate).
server.key	Si (secreto)	Solo en el broker — ruta habitual /etc/mosquitto/certs/server.key, legible unicamente por el usuario de servicio mosquitto	Llave privada del broker. NUNCA se transmite por la red: se usa localmente para firmar/derivar claves durante el handshake y asi demostrar la posesion del certificado (mensaje CertificateVerify).
password_file	Si (secreto)	Solo en el broker — ruta habitual /etc/mosquitto/password_file, generado con mosquitto_passwd	Verificadores de usuario/contrasena de la capa MQTT (usuario1/qwerty123). Es independiente de TLS: se usa despues del handshake, al procesar el paquete CONNECT.

server.crt, server.key y password_file NO forman parte de este repositorio de codigo (correcto por seguridad): viven unicamente en la Raspberry Pi que actua como broker. Su descripcion aqui se basa en la guia tecnica propia del proyecto (LaTeX-MQTT/REPORTE_MQTT_TLS_CIFRADO.tex) y en el contenido real del certificado incluido en el repositorio (CifradoSlave/certs/ca.crt). El unico archivo criptografico presente en el codigo analizado es ca.crt.

IMPORTANTE — NO es TLS mutuo (mTLS): el codigo solo entrega 'ca_certs' a tls_set() (sin certfile/keyfile de cliente), por lo que el broker NO exige ni verifica certificados de cliente y, por lo tanto, NO usa ca.crt en ningun momento. La identidad de Maestro y Esclavo ante el broker se resuelve unicamente con usuario y contrasena de la capa MQTT, protegidos por viajar dentro del tunel TLS ya establecido.

FASE I — TRANSPORTE: RESOLUCION DE NOMBRE Y CONEXION TCP (Maestro, puerto 8883)

1. Resolucion de nombre y apertura de socket TCP

'raspberrypiJED.local' se resuelve a una direccion IP local, habitualmente via mDNS (sufijo .local)
El maestro abre un socket TCP contra el puerto 8883 del broker
Saludo TCP de 3 vias: SYN -> SYN-ACK -> ACK
En este punto el canal ya es fiable y ordenado, pero AUN NO esta cifrado

FASE II — NEGOCIACION TLS (HANDSHAKE) PASO A PASO — Maestro <-> Broker

1) ClientHello

Version(es) TLS soportadas, lista de cipher suites, SNI = raspberrypiJED.local, valor aleatorio (random_cliente)

El broker localiza su material criptografico local

Lee de su propio sistema de archivos: server.crt (certificado) y server.key (llave privada)
NO consulta ca.crt en ningun momento: en este esquema, sin mTLS, el broker no verifica al cliente por certificado

2) ServerHello + Certificate

Version y cipher suite elegidas, random_broker, y el certificado server.crt (llave publica del broker + SAN=raspberrypiJED.local, firmado por CN=MQTT-Local-CA)

3) CertificateVerify — firma con server.key

El broker firma parte del intercambio con su llave privada server.key, demostrando que posee la llave asociada al certificado enviado. server.key nunca sale de la Raspberry Pi: solo viaja la firma resultante

4) ClientKeyExchange

El maestro genera un secreto de sesion y lo protege con la llave publica extraida de server.crt; con ese secreto ambos extremos derivan las mismas claves simetricas

5) ChangeCipherSpec + Finished

Ambos extremos confirman, ya bajo cifrado simetrico, que el canal quedo establecido correctamente

RESULTADO: tunel TLS activo sobre TCP/8883. Todo el trafico MQTT posterior (CONNECT, credenciales, PUBLISH, PUBACK) viaja cifrado dentro de este canal; un observador de la red solo ve bytes cifrados, nunca el contenido en claro.

FASE III — AUTENTICACION MQTT DENTRO DEL TUNEL TLS (Maestro <-> Broker)

CONNECT (usuario1 / qwerty123)

Las credenciales viajan cifradas dentro del tunel TLS ya establecido, nunca en texto plano sobre la red

El broker valida las credenciales

Compara usuario/contrasena contra su password_file interno (verificadores creados con mosquitto_passwd)
Este archivo pertenece a la capa de autenticacion MQTT y es independiente de ca.crt/server.crt/server.key (capa TLS)

CONNACK (rc=0, conexon aceptada)

FASE IV — PUBLICACION DEL MAESTRO (sesion TLS ya autenticada)

PUBLISH chaoskeystream/keys (QoS 1, retain=True)

payload: {ROSSLER_PARAMS(a,b,c), LOGISTIC_PARAMS(aLog,x0_log)} — viaja cifrado dentro del tunel de esta sesion

PUBACK

PUBLISH chaoskeystream/data (QoS 1, retain=True)

payload: {vector_cifrado[rmx], xyz/t_maestro[36000], ancho, alto, rmax, tiempo_sinc, KEYSTREAM} — igualmente cifrado en transito

PUBACK

Cierre de la sesion del Maestro

client.disconnect(): cierra el canal MQTT y termina la sesion TLS (cierre ordenado TCP/TLS)
Las claves simetricas negociadas en esta sesion se descartan; una futura conexon generara claves nuevas (secreto hacia adelante)

FASE V — RETENCION Y DISTRIBUCION EN EL BROKER

Persistencia por retencion (retain = True)

El broker termina la sesion TLS del Maestro en cada PUBLISH: obtiene el payload en claro y lo guarda en memoria/disco, asociado al topic
Cualquier cliente que se suscriba despues lo recibe de inmediato, sin esperar una nueva publicacion del Maestro
Al reenviarlo, el broker vuelve a cifrar ese mismo contenido bajo la sesion TLS propia de cada suscriptor (sesiones independientes, claves distintas)
QoS 1 garantiza entrega 'al menos una vez' mediante el ciclo PUBLISH / PUBACK

FASE VI — CONEXION DEL ESCLAVO: TCP + TLS (sesion independiente de la del Maestro)

1. Resolucion de nombre y apertura de socket TCP

Mismo procedimiento que en el Maestro: resolucion mDNS de 'raspberrypiJED.local' y apertura de socket TCP al puerto 8883

1)-5) Negociacion TLS completa (mismo procedimiento del handshake)

ClientHello -> ServerHello+Certificate -> CertificateVerify (firmada con server.key) -> verificacion local contra ca.crt propio -> ClientKeyExchange -> Finished

Es una sesion TLS totalmente independiente de la del Maestro: se reutilizan las mismas piezas del lado del broker (mismo server.crt / server.key, misma CA), pero se derivan claves de sesion efimeras distintas. El esclavo valida al broker contra su PROPIA copia local de ca.crt, ubicada en /home/tunchi/CifradoSlave/certs/ca.crt (ruta distinta a la del maestro, mismo contenido de CA).

CONNECT (usuario1 / qwerty123)

credenciales cifradas dentro del tunel TLS de esta sesion

El broker valida las credenciales del esclavo

Misma verificacion contra el password_file interno usado para el maestro (mismo usuario MQTT: usuario1)

CONNACK (rc=0) -> dispara on_connect()

SUBSCRIBE chaoskeystream/keys (QoS 1)

SUBSCRIBE chaoskeystream/data (QoS 1)

SUBACK x2

PUBLISH (mensaje retenido) chaoskeystream/keys

reenviado cifrado bajo la sesion TLS del esclavo — on_message_keys():
RECEIVED_KEYS = json.loads(payload)

PUBLISH (mensaje retenido) chaoskeystream/data
reenviado cifrado bajo la sesion TLS del esclavo — on_message_data():
RECEIVED_DATA = json.loads(payload)

Espera activa y cierre de conexon

while RECEIVED_KEYS is None or RECEIVED_DATA is None: time.sleep(0.2)
loop_stop() -> disconnect(): cierra el canal MQTT y termina esta segunda sesion TLS, independiente de la del maestro

Este diagrama cubre exclusivamente transporte TCP/TLS y capa MQTT (conexion, publicacion, retencion, suscripcion, recepcion). No incluye difusion/confusion ni sincronizacion caotica/descifrado de la imagen — ver el diagrama complementario "Diagrama_Comunicacion_MQTT_TLS".